

Asset lifecycle management for universities and studios — covering architecture, data models, API surface, and design rationale.

01

Problem Understanding

Universities, studios, and similar institutions manage shared physical assets — cameras, audio equipment, projectors, PA systems — across many users and time windows. Manual tracking via spreadsheets creates predictable failure modes: double-bookings, lost assets, no maintenance history, and no accountability trail.

Renex provides a structured asset lifecycle management system. Users can browse available inventory, request bookings, and track their loan history. Admins control the full approval-to-return workflow, log asset condition, and review analytics and audit logs.

Core Requirements

Multi-user roles	admin / user with enforced access control at every route
Asset inventory	Quantity-aware; images and QR codes per asset; soft delete
Booking lifecycle	pending → approved → issued → returned (or overdue)
Asset health	Condition log per asset; under_maintenance blocks availability
Notifications	In-app + email triggered on every lifecycle transition
Analytics	Usage trends, popular assets, status distribution (admin)
Audit trail	Immutable append-only log of all significant actions
Automated jobs	Daily overdue detection and due-date reminder cron jobs

System Architecture

Renex follows a standard three-tier architecture: a React SPA frontend, an Express REST API backend, and a MongoDB Atlas database. In production, Nginx acts as the sole public endpoint — serving the compiled SPA and proxying `/api/` and `/uploads/` to the backend container, keeping port 3000 entirely off the internet.

Layer	Technology	Responsibility
Client	React 18 · Vite · Tailwind CSS	Zustand for auth state · Axios with JWT interceptors
Proxy	Nginx (production only)	Serves SPA static files · proxies API + uploads
API	Express 5 · Node.js 20	REST endpoints · auth middleware · cron · Nodemailer
Database	MongoDB Atlas	6 collections · named Docker volume for uploads

Authentication Flow

Short-lived access tokens (15 min) travel in the `Authorization: Bearer` header — limiting the damage window if one is intercepted. A long-lived refresh token (7 days) lives in an `httpOnly` cookie, invisible to JavaScript and immune to XSS exfiltration. The Axios response interceptor silently fires `POST /auth/refresh` on any 401 and retries the original request, so the user never sees a forced logout unless their refresh token has also expired.

Middleware Stack

- `requireAuth` — verifies the JWT signature and attaches the decoded payload to `req.user`.
- `requireAdmin` — checks `role === 'admin'` in the payload; returns 403 otherwise.
- `node-cron` — two daily jobs: overdue detection at midnight and due-date reminders at 8 AM.
- **Nodemailer** — SMTP outbound email triggered on every booking lifecycle event.

Database Schema

Six MongoDB collections model the domain. All collections use Mongoose with timestamps unless noted. References are MongoDB ObjectIds. Indexes are noted in the field descriptions.

User

_id	ObjectId	Primary key
name	String	max 255
email	String	Unique, lowercase, indexed
passwordHash	String	bcrypt cost 12 — excluded from queries by default
role	String	admin user
isActive	Boolean	Soft deactivation. Indexed.

Asset

_id	ObjectId	Primary key
name / category / description	String	Compound full-text index
totalQuantity	Number	Immutable reference quantity
availableQuantity	Number	Live counter — decrements on booking, increments on reject/return
status	String	available low_stock unavailable — derived in pre-save hook
imageUrl	String	Relative path under /uploads/assets/
qrCode	String	Base64 data URL, generated on demand
isDeleted	Boolean	Soft delete — preserves referential integrity

Status is computed automatically: unavailable if ratio $\leq 0\%$, low_stock if $\leq 20\%$, otherwise available.

Booking

user	→ User	Requester
asset	→ Asset	Booked asset
quantityRequested	Number	min 1
purpose	String	Required — reason for the booking
startDate / endDate	Date	Requested date range
status	String	pending approved rejected issued returned overdue
adminNote	String	Optional note attached on approve/reject
approvedBy	→ User	Admin who actioned the booking
issueDetails	Object	issuedBy, issuedAt, dueDate, conditionAtIssue
returnDetails	Object	receivedBy, returnedAt, conditionAtReturn, damageNotes

AssetHealth

asset	→ Asset	Compound index with createdAt
condition	String	excellent good fair damaged under_maintenance
note	String	Free-text observation
reportedBy	→ User	Admin who logged the report
booking	→ Booking	Optional — links report to a return event

Notification

user	→ User	Recipient. Compound index with isRead.
title / message	String	Human-readable content
type	String	approval rejection due_reminder overdue issued returned
isRead	Boolean	Indexed with user for fast unread counts
relatedBooking	→ Booking	Optional deep link

AuditLog

performedBy	→ User	Indexed
action	String	e.g. BOOKING_APPROVED, ASSET_CREATED
entityType	String	asset booking user
entityId	ObjectId	Compound index with entityType
metadata	Mixed	Free-form snapshot — names, dates, notes at time of action
performedAt	Date	Default now. No updatedAt — documents are write-once.

Entity Relationships

User → Booking	1 : N	As requester (user field)
User → Booking	1 : N	As approver / issuer / receiver (separate refs)
Asset → Booking	1 : N	booking.asset
Asset → AssetHealth	1 : N	assetHealth.asset
Booking → Notification	1 : N	notification.relatedBooking
User → Notification	1 : N	notification.user
User → AuditLog	1 : N	auditLog.performedBy

API Overview

Base path `/api/v1`. All authenticated routes require `Authorization: Bearer <accessToken>`. Admin routes additionally enforce `role === 'admin'` in the token payload.

Auth — `/auth`

Method	Path	Access	Description
POST	<code>/register</code>	Public	Create account; returns access + refresh token pair
POST	<code>/login</code>	Public	Login; returns token pair
POST	<code>/logout</code>	Public	Clears refresh cookie
POST	<code>/refresh</code>	Public	Exchange refresh cookie for new access token
GET	<code>/me</code>	Auth	Current user profile
PUT	<code>/password</code>	Auth	Change password

Assets — `/assets`

Method	Path	Access	Description
GET	<code>/</code>	Public	Paginated list with full-text search, category and status filters
GET	<code>/:id</code>	Public	Asset detail
POST	<code>/</code>	Admin	Create asset — multipart/form-data, optional image upload
PUT	<code>/:id</code>	Admin	Update asset fields
DELETE	<code>/:id</code>	Admin	Soft delete (<code>isDeleted: true</code>)
GET	<code>/:id/qr</code>	Admin	QR code as base64 data URL
GET	<code>/:id/qr/download</code>	Admin	Download QR as PNG file
GET	<code>/:id/health</code>	Auth	Paginated condition history for this asset
POST	<code>/:id/health</code>	Admin	Log a condition report

Bookings — `/bookings`

Method	Path	Access	Description
POST	<code>/</code>	Auth	Create booking request
GET	<code>/mine</code>	Auth	Authenticated user's own bookings
GET	<code>/:id</code>	Auth	Booking detail — users restricted to own records
GET	<code>/</code>	Admin	All bookings, filterable by status / user / date
PUT	<code>/:id/approve</code>	Admin	Approve booking
PUT	<code>/:id/reject</code>	Admin	Reject + restore <code>availableQuantity</code>
PUT	<code>/:id/issue</code>	Admin	Mark issued, record due date and condition
PUT	<code>/:id/return</code>	Admin	Record return, restore inventory, log condition

Method	Path	Access	Description
POST	/bulk-approve	Admin	Approve multiple bookings by ID array
POST	/bulk-reject	Admin	Reject multiple + restore inventory

Analytics — /analytics (admin only)

Method	Path	Access	Description
GET	/summary	Admin	Stat cards: total assets, bookings, pending, overdue counts
GET	/utilization	Admin	Daily booking volume over the last 30 days
GET	/popular	Admin	Top 10 most-booked assets
GET	/status-distribution	Admin	Booking count grouped by status
GET	/category-utilization	Admin	Utilization rate per asset category
GET	/overdue	Admin	Current overdue bookings (paginated)

Notifications & Audit Logs

Method	Path	Access	Description
GET	/notifications	Auth	User's notifications — filterable, paginated
PUT	/notifications/:id/read	Auth	Mark a single notification as read
PUT	/notifications/read-all	Auth	Mark all notifications as read
GET	/audit-logs	Admin	Paginated log, filterable by entity type and user

Design Decisions

The following documents the non-obvious choices made during design and the reasoning behind each. Where trade-offs exist they are stated explicitly.

Dual-token JWT with httpOnly cookie

Access tokens are short-lived (15 min) and sent in the Authorization header — limiting the damage window if one is captured in transit. Refresh tokens live in an httpOnly cookie, making them inaccessible to JavaScript and therefore immune to XSS exfiltration. The Axios interceptor silently refreshes on any 401 and replays the original request, so the user never encounters a forced logout within a normal session.

sameSite: strict on the refresh cookie

With Vercel (frontend) and Render (backend) on different origins, sameSite: strict is intentional — refresh calls are issued from JavaScript (axios.post) and are treated as same-site only when both origins match. If cross-origin refresh is ever required, this setting must be relaxed to none with secure: true.

availableQuantity as a live denormalised field

Rather than computing availability from booking records at query time — which becomes expensive as booking volume grows — availableQuantity is maintained as a live counter on Asset. It decrements atomically when a booking is created and increments on rejection or return. A MongoDB session/transaction wraps the booking write and asset update to prevent race conditions under concurrent requests.

under_maintenance zeroes availability immediately

When an admin logs a health report with condition: under_maintenance, availableQuantity is set to 0, making the asset instantly unbookable without touching any existing bookings. Logging any other condition restores the full quantity. This provides a fast admin lever for taking assets offline while the booking system remains unaware.

Status derived in a pre-save hook

Asset status (available / low_stock / unavailable) is computed from availableQuantity / totalQuantity on every document save and stored as a plain field. The trade-off is a small write overhead in exchange for fast, index-friendly reads: status can be filtered directly without runtime arithmetic across the collection.

Soft delete on assets

Assets are never hard-deleted. Setting `isDeleted: true` hides an asset from all public listings while preserving referential integrity in existing bookings, health records, and audit logs. Historical data remains coherent; the asset can be reinstated without data loss.

Audit log is append-only

AuditLog documents carry no `updatedAt` field and expose no update routes — they are write-once by design. `performedAt` defaults to `Date.now` at insert time. This produces a tamper-resistant record of who did what and when, suitable for compliance and dispute resolution.

Cron jobs run in-process

The two daily jobs (overdue detection at midnight, due-date reminders at 8 AM) run inside the Express process via `node-cron` rather than as separate workers or cloud functions. This keeps the deployment footprint to a single container. The accepted trade-off is that scheduled jobs will not fire if the server is down at the scheduled time — acceptable for a non-critical reminder system, but worth revisiting if reliability requirements tighten.

Nginx as reverse proxy in production

In the Docker deployment, Nginx is the only container exposed to the internet. It serves the compiled SPA and proxies `/api/` and `/uploads/` to the backend on port 3000. From the browser's perspective everything is same-origin, which eliminates CORS entirely for self-hosted deployments. On Vercel + Render, where services are on different origins, CORS is handled explicitly via the `allowedOrigins` list in `app.js`.